

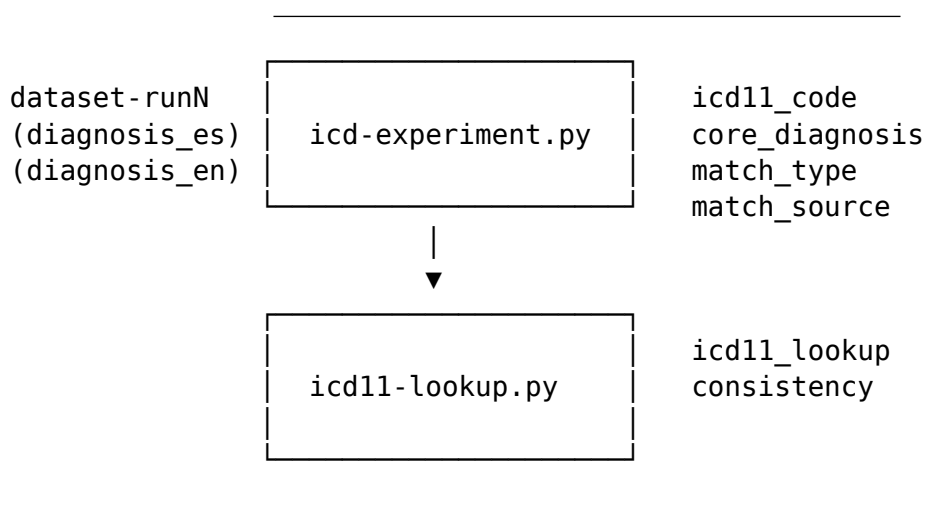
Pipeline de Etiquetado ICD-11

Estructura y flujo de icd-experiment.py e icd11-lookup.py

2026-07-21

Visión general

El pipeline opera en **dos fases secuenciales**, cada una implementada en un script independiente. El único artefacto de intercambio es el CSV del dataset, que ambos scripts modifican **in-place**.



icd-experiment.py — Fase 1: Asignación de código

Recursos y dependencias externas

Recurso	Función
en_product1.json (Orphanet)	KGraph: nombre/sinónimo → ICD-11
WHO ICD-11 API /search	Fallback semántico con flexisearch
nf_dictionary.json	Persistencia de casos no resueltos (NF-XXXX)
LM Studio — medgemma-27b-it	Identificación de enfermedad + validación semántica

Por qué se trabaja en inglés

- La API ICD-11 tiene más términos y sinónimos indexados en inglés.
- `en_product1.json` cubre 11 645 trastornos (6 553 con ICD-11) frente a 11 456/6 143 de la versión española.
- `diagnosis_es` se conserva solo para lectura humana y para la evaluación de coherencia posterior.

Construcción del KGraph (Orphanet)

Cada nodo almacena `canonical_name`, `orpha_code` e `icd11_codes`. Las claves del grafo son **nombres normalizados** (minúsculas, guiones como espacios, espacios colapsados). El nombre canónico tiene prioridad sobre los sinónimos: un sinónimo solo se indexa si su clave aún no existe.

Cascada de matching

La cascada opera íntegramente sobre `diagnosis_en`. Cada paso prueba **primero la frase completa** y después el nombre estándar identificado por el LLM (`core_diagnosis`), porque la frase completa es más específica y la causa puede determinar la categoría ICD-11 correcta.

Paso 0 — Identificación de la enfermedad (LLM)

Antes de cualquier búsqueda, el LLM recibe `diagnosis_en` y responde con el **nombre estándar de la enfermedad** tal como aparecería en una clasificación médica. No es un recorte de texto sino una identificación clínica:

"Cryptococcus neoformans meningoencephalitis" → "Cryptococcosis"

Si el LLM devuelve el mismo término (ya era el nombre canónico), la bandera `has_core` queda en `False` y los pasos siguientes solo prueban la frase completa.

Paso 1 — Caché en memoria

Evita recomputar el mismo diagnóstico si aparece varias veces en el CSV.

Paso 2 — Match exacto en KGraph

Busca la clave normalizada en el diccionario. Si hay nodo pero sin ICD-11 asociado, continúa al siguiente paso.

- Si resuelve con frase completa → `match_type = exact_complete`
- Si resuelve con nombre estándar → `match_type = exact_core`

Paso 3 — Match fuzzy en KGraph + validación LLM

Usa `rapidfuzz.token_sort_ratio` con umbral ≥ 85 . El candidato recuperado se valida semánticamente por el LLM **siempre contra la frase completa**, aunque el candidato haya venido del nombre estándar.

- `match_type = fuzzy_complete` o `fuzzy_core`

Paso 4 — WHO API /search + LLM

Lanza **dos consultas** a /search con flexisearch: una con la frase completa y otra con el nombre estándar. Los resultados se unifican en un pool deduplicado por código (la consulta completa tiene prioridad en caso de colisión). El LLM selecciona el candidato más adecuado.

Nota: /autocode fue descartado: en la corrida de referencia (201 casos) no resolvió ningún diagnóstico y sus propuestas coincidían con candidatos ya rechazados por /search.

- `match_type = who_search`
- `match_source = complete` o `core` (según qué consulta aportó el candidato elegido)

Paso 5 — NF (no encontrado)

Asigna un código NF-XXXX secuencial global, persistido en `nf_dictionary.json`. El mismo diagnóstico siempre recibe el mismo código entre corridas.

- `match_type = nf`, `match_source = none`

Ejecución y escritura

- Batches de **20 filas** procesadas con `asyncio.gather`.
- Escribe `icd11_code` (truncado a capítulo+categoría: "5A11.2" → "5A11"), `core_diagnosis`, `match_type` y `match_source`.
- El CSV se sobrescribe in-place al finalizar, conservando todas las columnas originales.

icd11-lookup.py — Fase 2: Búsqueda inversa y coherencia

Fase 1 — Búsqueda inversa: código → título oficial (sin LLM)

Para cada fila con `icd11_code`:

Códigos NF-XXXX Se resuelven localmente contra `nf_dictionary.json`. El resultado es "[NF] etiqueta".

Códigos ICD-11 reales Se consulta `/codeinfo/{code}`, que devuelve un `stemId` (URI canónica de la entidad). Se sigue ese URI para obtener el título oficial. Si `/codeinfo` falla, el fallback es `/search` restringido al código exacto.

Se ejecutan **8 workers concurrentes** (`ThreadPoolExecutor`) con caché en memoria por código.

Fase 2 — Evaluación de coherencia (LLM)

El LLM recibe `diagnosis_es` e `icd11_lookup` y responde `yes` o `no`:

¿El título de la categoría ICD-11 es coherente con el diagnóstico?

Criterio flexible: los códigos están truncados a capítulo+categoría, por lo que el título siempre es más general que el diagnóstico. Que englobe correctamente al diagnóstico cuenta como `yes`.

Situación	Resultado
Código NF (por defecto)	<code>consistency = ""</code> (omitido)
Código ICD-11 sin lookup	<code>consistency = "no"</code>
LLM confirma coherencia	<code>consistency = "yes"</code>
LLM niega coherencia	<code>consistency = "no"</code>
Error o respuesta inválida del LLM	<code>consistency = ""</code>

Las filas NF se omiten por defecto porque su lookup es el propio diagnóstico, lo que daría `yes` trivialmente e inflaría la métrica. El flag `--eval-nf` las incluye.

Batches de **10 filas** con `asyncio.gather`.

Opciones de línea de comandos

Flag	Efecto
<code>--overwrite</code>	Rehace filas ya procesadas
<code>--skip-consistency</code>	Solo fase 1, sin invocar el LLM
<code>--eval-nf</code>	Evalúa coherencia también en filas NF
<code>--rows N</code>	Procesa solo las primeras N filas (modo prueba)
<code>--workers N</code>	Concurrencia WHO API (default: 8)
<code>--lang es en</code>	Idioma de los títulos devueltos por la API

Columnas escritas

Columna	Contenido
<code>icd11_lookup</code>	Título oficial WHO en español (o [NF] etiqueta)
<code>consistency</code>	<code>yes no ""</code>

Artefactos persistentes entre corridas

Archivo	Contenido	Compartido entre scripts
dataset-runN.csv	Dataset principal, modificado in-place	Sí (salida de fase 1, entrada de fase 2)
nf_dictionary.json	Mapa diagnóstico NF → código NF-XXXX	Sí (escrito por fase 1, leído por fase 2)

nf_dictionary.json garantiza que el mismo diagnóstico no resuelto recibe el mismo código en todas las corridas, lo que hace posible calcular la **estabilidad de asignación** entre runs.